



University of Engineering & Technology Lahore

Narowal campus

Department of Computer Sciences and Engineering

LAB MANUALS:

Artificial Intelligence

Submitted by:

HUSNAIN AZAM 2024-CS -520

MUHAMMAD AWAIS 2024-CS-589

Submitted to:

Mam Rabia Sana

Lab 01

Installation and Configuration of Anaconda Environment

Abstract

The purpose of this laboratory session was to install and configure a scientific computing environment suitable for Artificial Intelligence (AI) experimentation. The Anaconda distribution was selected due to its integrated package management system and compatibility with AI-related libraries. The lab focused on downloading, installing, and configuring Anaconda on a Windows-based system, managing virtual environments using Conda, and launching Jupyter Notebook for interactive development. The successful completion of this lab established the foundational software infrastructure required for future AI experiments.

1. Introduction

Artificial Intelligence development requires a stable and well-configured computational environment. Installing individual programming libraries separately often leads to dependency conflicts and compatibility issues. To address these challenges, integrated distributions such as Anaconda are widely adopted in academia and industry.

Anaconda provides a comprehensive platform for scientific computing, including package management, environment isolation, and integrated development tools. This lab session was designed to familiarize students with the installation process and basic configuration of this environment.

2. Background and Theoretical Framework

2.1 Scientific Computing Environments

AI systems rely on multiple libraries for data manipulation, numerical computation, visualization, and model development. Proper environment configuration ensures:

- Dependency management
- Version control
- Reproducibility of experiments
- System stability

Without structured environment management, conflicts between library versions may lead to runtime errors and unreliable results.

2.2 Overview of Anaconda Distribution

Anaconda is an open-source distribution designed for large-scale data processing and computational research. It includes:

- Conda package manager
- Environment management tools
- Pre-configured scientific libraries
- Graphical interface (Anaconda Navigator)
- Command-line interface (Anaconda Prompt)

The inclusion of Conda enables isolated virtual environments, which are essential for managing different AI projects independently.

2.3 Interactive Development with Jupyter Notebook

Jupyter Notebook is a browser-based interactive computational environment. It supports:

- Cell-based execution
- Integrated documentation
- Kernel-based execution control
- Real-time output visualization

In AI research, Jupyter Notebook enhances experimentation by enabling incremental testing and documentation within the same interface.

3. System Requirements

Prior to installation, system compatibility was verified. The minimum requirements included:

- Operating System: Windows 10 / Windows 11 (64-bit architecture)
- RAM: Minimum 4 GB
- Storage: At least 3 GB free disk space
- Internet connectivity for installer download

The system architecture was confirmed via system properties to ensure compatibility with the 64-bit installer.

4. Methodology

4.1 Download Procedure

The following steps were performed:

- Accessed the official website of Anaconda using a web browser.
- Navigated to the download section.

- Selected the Windows 64-bit graphical installer.
- Downloaded the executable installation file.

4.2 Installation Procedure

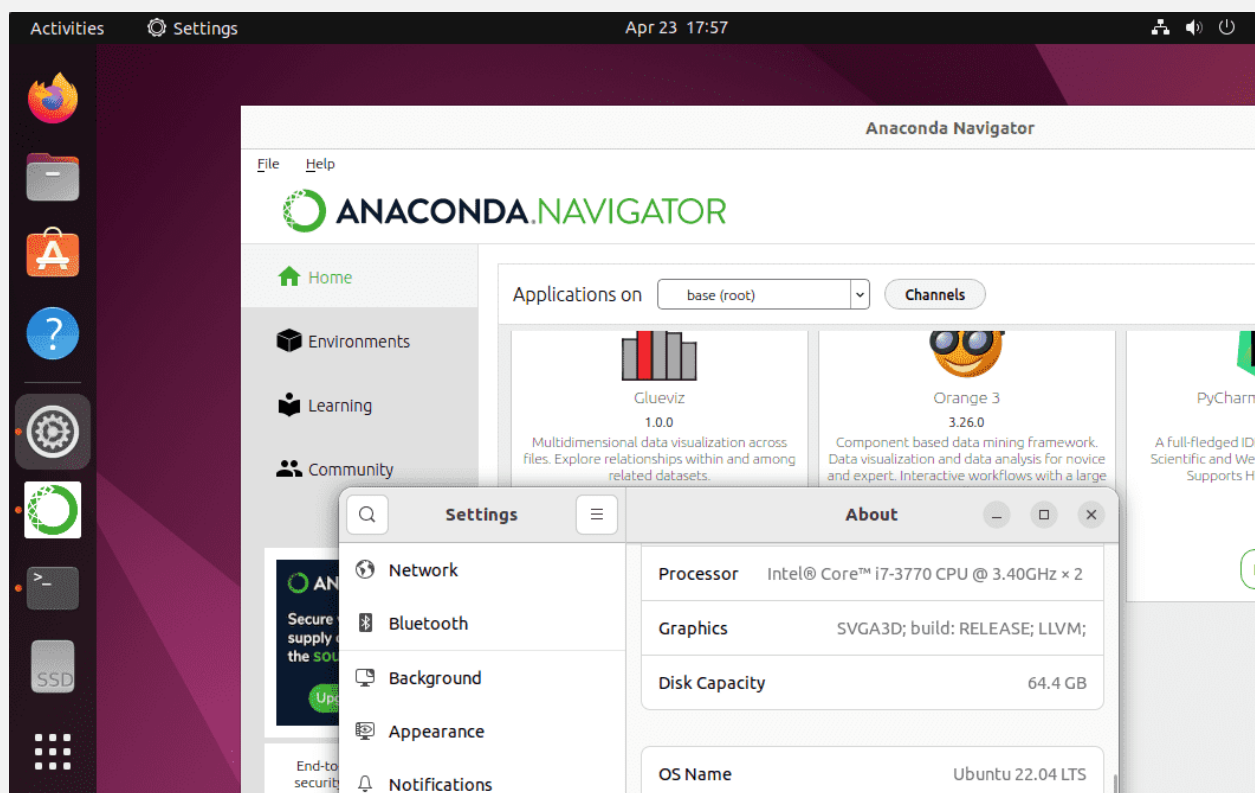
The installation process involved the following structured steps:

Anaconda Navigator

Upon successful installation, Anaconda Navigator was launched. This graphical interface allows users to:

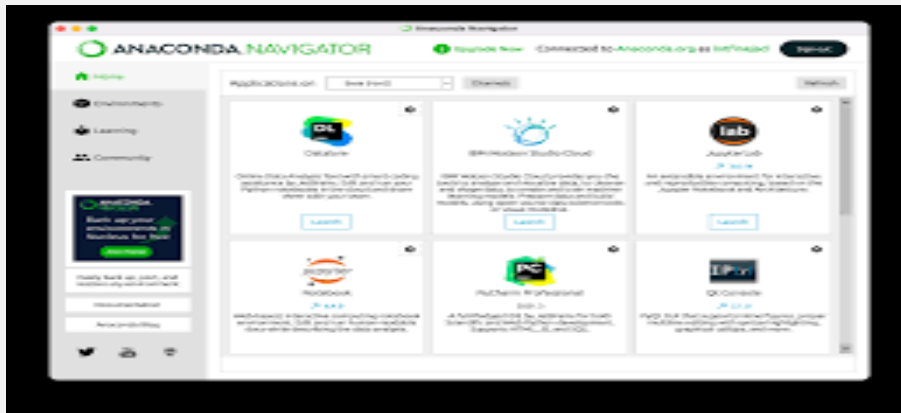
- Manage software environments
- Install and update packages
- Launch development tools
- Monitor system configuration

Navigator simplifies environment management compared to manual command-line configuration.



Step 3: Installation Directory

The default directory path was retained to avoid configuration inconsistencies.



Step 4: Advanced Configuration Options

The option to register Anaconda as the default Python distribution was selected. The option to modify system PATH variables was not enabled to prevent potential conflicts.

Step 5: Completion

The installation process was completed successfully, and confirmation was received from the setup wizard.



Step 1: Execution of Installer

The downloaded .exe file was executed to initiate the setup wizard.

Step 2: Selection of Installation Type

The “Just Me (Recommended)” option was selected to install the software for the current user only.

5.2 Conda Environment Management

Conda facilitates the creation of isolated virtual environments. The following procedure was followed:

- Navigated to the “Environments” tab in Navigator.
- Selected “Create New Environment.”
- Assigned an environment name.
- Selected the desired Python version.
- Completed environment creation.

This ensured project-level isolation and minimized dependency conflicts.

6. Launching Jupyter Notebook

Jupyter Notebook was launched using two methods:

Method 1: Through Anaconda Navigator

The “Launch” button under Jupyter Notebook was selected.

Method 2: Through Anaconda Prompt

The command:

```
jupyter notebook
```

was executed, opening the interface in the default web browser.

7. Interface Exploration

The Jupyter Notebook dashboard provides:

- File management system
- Notebook creation interface
- Kernel management options
- Save, export, and shutdown functions

Each notebook operates using a computational kernel responsible for executing instructions. Kernel controls include restart, interrupt, and shutdown options.

8. Results

The lab objectives were successfully achieved:

- Anaconda distribution was properly installed.
- Environment configuration was completed.

Lab 02

Strings and Their Syntax

Abstract

This laboratory experiment focused on the study and practical implementation of Python string manipulation techniques. The objective was to explore built-in string methods and apply them in the development of a structured text-based user profile management system. The lab emphasized input validation, formatting, transformation, alignment, encryption, and verification using Python's string handling capabilities. The final system demonstrated how string operations can be utilized to ensure data integrity, user interaction, and formatted output in command-line applications.

1. Introduction

String processing is a fundamental aspect of programming, especially in applications that involve user input and text-based interaction. In Python, strings are immutable sequences of Unicode characters and provide a wide range of built-in methods for manipulation and validation.

This lab aimed to strengthen understanding of string syntax and demonstrate their practical application by developing a command-line user profile system. Each stage of the system utilized specific string methods for validation, formatting, and transformation.

2. Objectives

The objectives of this lab were:

- To understand the syntax and behavior of Python string methods.
- To apply string validation techniques in real-world scenarios.
- To implement a structured user profile system.
- To format and present textual data professionally.
- To perform basic encryption and data masking.

3. Theoretical Background

Python provides numerous string methods that operate on string objects and return modified versions of the original string. Since strings are immutable, these methods return new strings rather than modifying the original data.

Key categories of string operations explored in this lab include:

- Case conversion methods
- Validation methods
- Searching and indexing methods
- Formatting and alignment methods

- Splitting and joining methods
- Replacement and translation methods
- Encoding methods

These operations are essential in text-based systems such as login systems, data validation tools, and profile management platforms.

4. Methodology and Implementation

The lab task required the development of a structured text-based user profile system. The implementation was divided into logical modules as described below.

4.1 User Registration Module

In this stage, the user was prompted to enter:

- First name
- Last name
- Username

Methods Used

- `capitalize()` – Ensured the first letter of names was uppercase.
- `isalpha()` – Verified that first and last names contained only alphabetic characters.
- `isalnum()` – Validated that the username contained only letters and numbers.

Functional Behavior

- If names contained non-alphabetic characters, the system displayed an error message.
- Valid names were automatically capitalized.
- If the username contained invalid characters (e.g., special symbols), the system requested correction.

4.2 Password Setup Module

The system required users to create a secure password.

Methods Used

- `islower()`
- `isupper()`
- `isdigit()`
- `isspace()`
- `swapcase()`

Functional Behavior

- Password was checked for at least one lowercase letter.

- Password was checked for at least one uppercase letter.
- Password was checked for at least one digit.
- If spaces were detected using `isspace()`, the user was prompted to remove them.
- `swapcase()` was optionally used to analyze character strength patterns.

This module ensured basic password complexity validation.

```

: password = input("Create a password: ")

has_lower = False
has_upper = False
has_digit = False
has_space = False

for char in password:
    if char.islower():
        has_lower = True
    elif char.isupper():
        has_upper = True
    elif char.isdigit():
        has_digit = True
    elif char.isspace():
        has_space = True

if has_space:
    print("Password should not contain spaces. Please remove them.")

if has_lower and has_upper and has_digit and not has_space:
    print("Password is strong and valid")
else:
    print("Password is weak")
    print("It must contain at least one lowercase letter, one uppercase letter, one digit, and no spaces")
swapped_password = password.swapcase()
print("Swapped case version of your password:", swapped_password)

Create a password: Ayesha1
Password is strong and valid
Swapped case version of your password: aYESHA1

```

4.3 Profile Description Module

Users entered a short description about themselves.

Methods Used

- `center(width, fillchar)`
- `count()`
- `replace()`

Functional Behavior

- The description was centered within 50 characters using hyphens (-).
- The system counted occurrences of a specific word (e.g., “I”).
- Any occurrence of the word “boring” was replaced with “exciting”.

This demonstrated formatting and text transformation capabilities.

```
password = input("Enter your password: ")

has_lower = False
has_upper = False
has_digit = False
has_space = False

for ch in password:
    if ch.islower():
        has_lower = True
    elif ch.isupper():
        has_upper = True
    elif ch.isdigit():
        has_digit = True
    elif ch.isspace():
        has_space = True

if has_space:
    print("Password contains spaces. Please remove them.")

if has_lower and has_upper and has_digit and not has_space:
    print("Password is valid")
else:
    print("Password is invalid")

print("Swapped case password:", password.swapcase())

description = input("\nEnter a brief description about yourself: ")

centered_description = description.center(50, "-")
print("\nCentered Description:")
```

```
centered_description = description.center(50, "-")
print("\nCentered Description:")
print(centered_description)

word_count = description.count("I")
print("Number of times 'I' appears:", word_count)

updated_description = description.replace("boring", "exciting")
print("Updated Description:")
print(updated_description)
```

```
Enter your password: ayesha 1
Password contains spaces. Please remove them.
Password is invalid
Swapped case password: AYESHA 1
```

```
Enter a brief description about yourself: I'm not a boring girl
```

```
Centered Description:
-----I'm not a boring girl-----
Number of times 'I' appears: 1
Updated Description:
I'm not a exciting girl
```

4.4 User Preferences Module

Users entered hobbies separated by commas.

Methods Used

- split()
- join()

Functional Behavior

- Input string was split into a list of hobbies using commas as separators.
- The hobbies were rejoined into a formatted string separated by vertical bars (|).

This module demonstrated list conversion and formatted output generation.

```
hobbies_input = input("Enter your favorite hobbies (separated by commas): ")

hobbies_list = hobbies_input.split(",")

formatted_hobbies = " | ".join(hobbies_list)

print("Your hobbies are:")
print(formatted_hobbies)
```

Enter your favorite hobbies (separated by commas): painting,sketching,dancing,cooking
Your hobbies are:
painting | sketching | dancing | cooking

4.5 Account Verification Module

This module validated email formatting and verification codes.

Methods Used

- find()
- replace()
- startswith()
- endswith()
- zfill()

Functional Behavior

- The index of “@” was located using find().
- Characters before “@” were masked for privacy.
- Email prefix and domain were validated using startswith() and endswith().
- Numeric verification codes were padded to 5 digits using zfill().

This demonstrated string searching, masking, and formatting techniques.

```

email = input("Enter your email address: ")
at_index = email.find("@")
masked_email = email.replace(email[at_index:], email[0] + "****")
print("Masked Email:", masked_email)

confirm = input("Is this your email? (yes/no): ")

if email.startswith("j"):
    print("Email starts with the letter 'j'")

if email.endswith(".com"):
    print("Email ends with .com")

code = input("Enter verification code: ")
formatted_code = code.zfill(5)

print("Your 5-digit verification code is:", formatted_code)

```

```

Enter your email address: ayesha1232gmail.com
Masked Email: a****m
Is this your email? (yes/no): y
Email ends with .com
Enter verification code: 1123
Your 5-digit verification code is: 01123

```

4.6 Profile Summary Display

The system displayed user information in a structured format.

Methods Used

- ljust()
- rjust()
- title()
- upper()

Functional Behavior

- Section headings were displayed in uppercase.
- Data fields were aligned using left and right justification.
- All user text was formatted into title case.

This improved readability and professional presentation.

```

first_name = "ayesha"
last_name = "qaiser"
username = "ayesha1"
description = "I am an exciting student"
hobbies = "painting | sketching | dancing | cooking"
email = "ayesha123@gmail.com"

```

```

print("\n" + "PROFILE SUMMARY".upper())
print("-" * 50)

# Display user details in columns
print("First Name:".ljust(15), first_name.title())
print("Last Name:".ljust(15), last_name.title())
print("Username:".ljust(15), username)
print("Description:".ljust(15), description.title())
print("Hobbies:".ljust(15), hobbies.title())
print("Email:".ljust(15), email)

print("-" * 50)

```

```

PROFILE SUMMARY
-----
First Name:    Ayesha
Last Name:     Qaiser
Username:      ayesha1
Description:    I Am An Exciting Student
Hobbies:       Painting | Sketching | Dancing | Cooking
Email:         ayesha123@gmail.com
-----

```

4.7 User Feedback Module

Users provided feedback about the platform.

Methods Used

- expandtabs()
- strip()
- lstrip()
- rstrip()
- istitle()

Functional Behavior

- Tabs were expanded into spaces.
- Leading and trailing whitespace was removed.
- Feedback was checked for title case format.

This ensured clean and properly formatted input.

4.8 Encryption and Encoding Module

Sensitive user data was encrypted.

Methods Used

- maketrans()
- translate()
- encode()

Functional Behavior

- A translation table was created.
- Specific characters were replaced with encoded equivalents.
- Data was optionally encoded into UTF-8 format.

This demonstrated basic text encryption and encoding techniques.

```
user_data = input("Enter some sensitive data to encrypt: ")
```

```
translation_table = str.maketrans({  
    'a': '@',  
    'e': '3',  
    'i': '!',  
    'o': '0',  
    's': '$'  
})
```

```
encrypted_data = user_data.translate(translation_table)  
print("Encrypted Data:", encrypted_data)
```

```
utf8_encoded = encrypted_data.encode('utf-8')  
print("UTF-8 Encoded Data:", utf8_encoded)
```

```
Enter some sensitive data to encrypt: happy anniversary ayesha bhabhi and zeeshan bhai  
Encrypted Data: h@ppy @nn!v3r$h@ry @y3$h@ bh@bh! @nd z33$h@n bh@!  
UTF-8 Encoded Data: b'h@ppy @nn!v3r$h@ry @y3$h@ bh@bh! @nd z33$h@n bh@!'
```

4.9 Profile Completion Status

Methods Used

- max()
- min()

Functional Behavior

- The lexicographically highest and lowest characters from the user's name were identified.
- These characters were combined to generate a unique profile completion code.

This showcased character comparison operations.

Bonus Task:

Implement error handling for invalid inputs and edge cases. • Allow users to update their profile information, using the appropriate string methods to validate and format the updated data.

```
1.] def get_valid_name(prompt):
    while True:
        name = input(prompt).strip()
        if not name.isalpha():
            print("Name must contain only letters. Please try again.")
        else:
            return name.capitalize()

def get_valid_username():
    while True:
        username = input("Enter a username: ").strip()
        if not username.isalnum():
            print("Username must contain only letters and numbers.")
        else:
            return username

def get_valid_password():
    while True:
        password = input("Create a password: ").strip()
        if any(ch.isspace() for ch in password):
            print("Password should not contain spaces.")
            continue
        if not any(ch.islower() for ch in password):
            print("Password must contain at least one lowercase letter.")
            continue
        if not any(ch.isupper() for ch in password):
            print("Password must contain at least one uppercase letter.")
            continue
        if not any(ch.isdigit() for ch in password):
            print("Password must contain at least one digit.")
            continue
```

```

        if not any(ch.isdigit() for ch in password):
            print("Password must contain at least one digit.")
            continue
        return password

def get_profile_description():
    desc = input("Enter a brief description about yourself: ").strip()
    desc = desc.replace("boring", "exciting")
    return desc.center(50, "-")

def get_hobbies():
    hobbies_input = input("Enter your favorite hobbies (comma separated): ").strip()
    hobbies_list = [h.strip() for h in hobbies_input.split(",") if h.strip()]
    return " | ".join(hobbies_list)

def get_email():
    while True:
        email = input("Enter your email: ").strip()
        at_index = email.find("@")
        if at_index == -1:
            print("Invalid email. Must contain '@'.")
            continue
        masked_email = email[0] + "****" + email[at_index:]
        print("Masked Email:", masked_email)
        if not email.startswith("j"):
            print("Warning: email does not start with 'j'.")
        if not email.endswith(".com"):
            print("Warning: email does not end with .com.")
        return email, masked_email

def get_feedback():

```

```

def get_email(prompt):
    while True:
        email = input(prompt).strip()
        if not email:
            print("Email must contain @")
            return None
        return email

def get_valid_username():
    while True:
        username = input("Enter a valid username: ")
        if not username:
            print("Username must contain at least 3 characters")
            return None
        return username

def get_valid_password():
    while True:
        password = input("Enter a valid password: ")
        if not password:
            print("Password must contain at least 8 characters")
            return None
        return password

```

Snipping Tool

Screenshot copied automatically

```

def get_feedback():
    feedback = input("Rate your experience: ").expandtabs().strip()
    if not feedback.istitle():
        print("Please rephrase feedback in title case.")
    return feedback

def encrypt_data(data):
    table = str.maketrans({'a': '@', 'e': '3', 'i': '!', 'o': '0', 's': '$'})
    encrypted = data.translate(table)
    utf8_encoded = encrypted.encode('utf-8')
    return encrypted, utf8_encoded

def profile_completion_code(name):
    highest = max(name)
    lowest = min(name)
    return highest + lowest

def display_profile(user):
    print("\n" + "PROFILE SUMMARY".upper())
    print("-"*50)
    print("First Name:".ljust(15), user['first_name'].title())
    print("Last Name:".ljust(15), user['last_name'].title())
    print("Username:".ljust(15), user['username'])
    print("Description:".ljust(15), user['description'].title())
    print("Hobbies:".ljust(15), user['hobbies'].title())
    print("Email:".ljust(15), user['masked_email'])
    print("Feedback:".ljust(15), user['feedback'])
    print("Encrypted Data:".ljust(15), user['encrypted'])
    print("Profile Code:".ljust(15), user['completion_code'])
    print("-"*50)

```



```

print("\n")

def main():
    user = {}
    user['first_name'] = get_valid_name("Enter your first name: ")
    user['last_name'] = get_valid_name("Enter your last name: ")
    user['username'] = get_valid_username()
    user['password'] = get_valid_password()
    user['description'] = get_profile_description()
    user['hobbies'] = get_hobbies()
    user['email'], user['masked_email'] = get_email()
    user['feedback'] = get_feedback()
    user['encrypted'], user['utf8'] = encrypt_data(user['first_name'] + user['last_name'])
    user['completion_code'] = profile_completion_code(user['first_name'] + user['last_name'])

    display_profile(user)

    # Allow user to update profile
    update = input("\nDo you want to update your profile? (yes/no): ").strip().lower()
    if update == "yes":
        user['first_name'] = get_valid_name("Update first name: ")
        user['last_name'] = get_valid_name("Update last name: ")
        user['username'] = get_valid_username()
        display_profile(user)

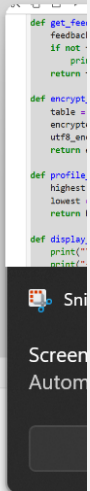
main()

```

```

Enter your first name: ayesha
Enter your last name: qaiser
Enter a username: ayesha1
Create a password: ayesHa1

```



```

Enter your first name: ayesha
Enter your last name: qaiser
Enter a username: ayesha1
Create a password: ayesHa1
Enter a brief description about yourself: I'm an artist
Enter your favorite hobbies (comma separated): painting,dancing,cooking
Enter your email: ayesha1@gmail.com
Masked Email: a****@gmail.com
Warning: email does not start with 'j'.
Rate your experience: that was good
Please rephrase feedback in title case.

```

PROFILE SUMMARY

```

-----
First Name:      Ayesha
Last Name:       Qaiser
Username:        ayesha1
Description:      -----I'M An Artist-----
Hobbies:         Painting | Dancing | Cooking
Email:           a****@gmail.com
Feedback:        that was good
Encrypted Data:  Ay3$h@Q@!$3r
Profile Code:    yA
-----

```

LAB#03

1. Conditional Statements (if-else)

Conditional statements control the flow of a program depending on certain conditions. They allow programs to make decisions dynamically.

Key Points:

- if checks a condition; if it is true, the block executes.
- elif (else-if) checks another condition if the previous if was false.
- else executes when all previous conditions are false.

Nested Conditions:

You can place an if inside another if to check multiple levels of conditions.

Ternary Operator:

A shorthand for simple if-else statements, often used for assigning values based on a condition.

2. Loops

Loops allow repeated execution of code based on a condition.

2.1 for Loop

- Iterates over a sequence (list, tuple, string, dictionary keys/values).
- Often used with range() to iterate a specific number of times.
- Can include an optional else block executed if the loop completes normally.

2.2 while Loop

- Executes as long as a given condition is true.
- Can also have an optional else block executed when the condition becomes false.
- break terminates the loop immediately.
- continue skips the current iteration and moves to the next.

Nested Loops

Loops can be placed inside other loops for multi-dimensional iteration. Useful in processing grids, matrices, or combinations of sequences.

3. Tuples

Tuples are **immutable sequences** of elements.

Key Concepts:

Once created, tuple elements cannot be changed. Support indexing, slicing, and nesting. Can store multiple data types. Can be concatenated or repeated. Useful for storing data that should not be modified, like fixed records or coordinates.

Operations:

Slicing to access parts of the tuple. Length to count elements. Converting from lists or strings to tuples for immutable storage.

4. Lambda Functions

Lambda functions are **anonymous, one-line functions**.

Key Concepts:

Defined without a name. Useful for short operations that are not reused elsewhere. Often used with functions like `map()`, `filter()`, and `reduce()`. Can be returned from other functions to create customizable behavior (higher-order functions).

5. Dictionaries

Dictionaries store **key-value pairs** and allow fast access to values by their keys.

Key Concepts:

Keys must be unique; values can repeat. Dictionaries are unordered (Python 3.7+ preserves insertion order but still considered unordered logically). Common operations: create, update, delete, retrieve values. Useful for mapping relationships, storing configurations, or keeping records.

Important Methods:

- `get()` – safely access a value.
- `update()` – add or modify multiple entries.
- `pop()` / `popitem()` – remove entries.
- `keys()`, `values()`, `items()` – inspect dictionary contents.

6. `os.link()`

Creates a **hard link** pointing to an existing file. A hard link acts like another name for the same file. Useful for file management, backup, or file system organization. Does not copy data; multiple links refer to the same physical content.

LAB TASK:

The Student Grades Management System is designed to efficiently manage and process student grades. It involves several tasks:

1. Storing Student Data:

o Tuples are used to store each student's name and their corresponding grade. Each tuple contains two elements: the student's name and their numerical grade. For example, a tuple might look like ('Alice', 85).

2. Evaluating Grades:

o Conditional Statements are utilized to convert numerical grades into letter grades (A, B, C, D, F). This is done by defining specific grade ranges. For instance, a grade of 85 would be classified as a 'B', while a grade of 92 would be classified as an 'A'.

3. Generating Reports:

o Loops are employed to iterate over the student data. During this process, the system. Generates reports based on the classified grades. It prints out each student's name. Along with their letter grade. Additionally, loops can be used to check and report.

Performance levels, such as noting students who excel or need improvement.

```
[1]: students = [
    ('Ayesha', 85),
    ('Bisma', 92),
    ('Arsalan', 67),
    ('Noman', 74),
    ('Muhammad', 58)
]

def get_letter_grade(marks):
    if marks >= 90:
        return 'A'
    elif marks >= 80:
        return 'B'
    elif marks >= 70:
        return 'C'
    elif marks >= 60:
        return 'D'
    else:
        return 'F'

print("----- Student Report -----")
grade_dict = {}
for name, marks in students:
    letter = get_letter_grade(marks)
    grade_dict[name] = letter
    print(f"{name} got grade {letter}")

if letter == 'A':
    print("    Excellent Performance!")
elif letter == 'F':
    print("    Needs Improvement!")
print("\n----- Updating Grades -----")
```

4.

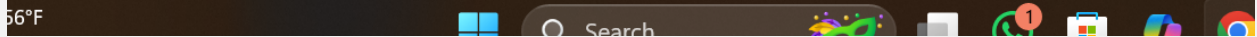
```
print(f"{name} got grade {letter}")

if letter == 'A':
    print("    Excellent Performance!")
elif letter == 'F':
    print("    Needs Improvement!")
print("\n----- Updating Grades -----")
student_name = input("Enter student name to update grade: ")
if student_name in grade_dict:
    new_marks = int(input("Enter new numerical grade: "))
    new_letter = get_letter_grade(new_marks)
    grade_dict[student_name] = new_letter
    print(f"{student_name}'s grade updated to {new_letter}")
else:
    print("Student not found!")
print("\n----- Final Updated Report -----")
for name, letter in grade_dict.items():
    print(f"{name}: {letter}")

----- Student Report -----
Alice got grade B
Bob got grade A
    Excellent Performance!
Charlie got grade D
David got grade C
Emma got grade F
    Needs Improvement!

----- Updating Grades -----
Enter student name to update grade: Alice
Enter new numerical grade: 83
Alice's grade updated to B
```

5.



LAB #04

Objective

- Understand and implement the **linear search algorithm**.
- Perform linear search on arrays of integers, strings, and data read from files.

1. Introduction to Linear Search

Linear Search is the simplest searching algorithm. It checks each element of a sequence (array, list, or file) **sequentially** until the target element is found or the end of the sequence is reached.

Characteristics:

- **Time Complexity:** $O(n)$ → depends on the number of elements n .
- **Space Complexity:** $O(1)$ → search is performed in-place without extra memory.
- **Advantages:** Easy to implement, works on unsorted data.
- **Disadvantages:** Inefficient for large datasets.

2. Steps for Linear Search

1. Start at the first element of the array or list.
2. Compare the current element with the target element.
3. If it matches, return the position (index) of the element.
4. If not, move to the next element.
5. Repeat until the target is found or the end is reached.

3. Linear Search Applications

3.1 Searching in an Array of Integers

- Check each integer sequentially until the target integer is found.
- Can be adapted to take input from the user.
- Often used for small datasets or unsorted arrays.

3.2 Searching in an Array of Strings

- Similar to integer search, but compares string values.
- Useful for searching names, words, or labels in an array.

3.3 Searching in Files

- Can search for a word, sentence, or record in a text file.
- Reads file line by line and compares each line with the target.

3.4 Searching Records in Files (CSV)

- Each line may contain a record (e.g., student name or ID).
- Linear search scans all lines sequentially until the target record is found.
- Useful for small to medium-sized files; for large files, more advanced techniques are recommended.

LAB TASK:

Implement binary search as all the above mentioned examples and compare it with linear search on sorted arrays. Discuss the time complexity difference.

```
•[2]: def binary_search_integers(arr, target):
    left = 0
    right = len(arr) - 1
    while left <= right:
        mid = (left + right) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1

    return -1
arr = [10, 25, 30, 45, 50]
target = 30
result = binary_search_integers(arr, target)
if result != -1:
    print(f"Element {target} found at index {result}")
else:
    print("Element not found")
```

Element 30 found at index 2

```
[3]: def binary_search_strings(arr, target):
    left = 0
    right = len(arr) - 1
    while left <= right:
        mid = (left + right) // 2

        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1

    return -1
arr = ["apple", "banana", "cherry", "date"]
target = "cherry"
result = binary_search_strings(arr, target)
if result != -1:
    print(f"Element '{target}' found at index {result}")
else:
    print("Element not found")
```

Element 'cherry' found at index 2

```
[11]: def binary_search_file(file_path, target):
    try:
        with open(file_path, 'r') as file:
            lines = file.readlines()
            lines = [line.strip() for line in lines]
            lines.sort()
            left = 0
            right = len(lines) - 1
            while left <= right:
                mid = (left + right) // 2
                if target == lines[mid]:
                    return mid
                elif target > lines[mid]:
                    left = mid + 1
                else:
                    right = mid - 1
            return -1
    except FileNotFoundError:
        print("File not found")
        return -1

file_path = "untitled.txt"
target = "apple"
result = binary_search_file(file_path, target)
if result != -1:
    print(f"Word '{target}' found on line {result + 1}")
else:
    print(f"Word '{target}' not found in the file")
```

Word 'apple' found on line 1

```
def binary_search_integers(arr, target):
    arr.sort() # Sort first
    left, right = 0, len(arr)-1
    while left <= right:
        mid = (left + right)//2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1

arr = [10, 25, 30, 45, 50]
target = int(input("Enter integer to search: "))
print("Binary Search:", binary_search_integers(arr, target))
```

Enter integer to search: 20
Binary Search: -1

```
def binary_search_strings(arr, target):
    arr.sort()
    left, right = 0, len(arr)-1
    while left <= right:
        mid = (left + right)//2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1

arr = ["apple", "banana", "cherry", "date", "plum"]
target = input("Enter string to search: ")
print("Binary Search:", binary_search_strings(arr, target))
```

```
Enter string to search: banana
Binary Search: 1
```


LAB #05

Depth First Search (DFS) and Breadth First Search (BFS) using Graphs Objective

1. Understand graph traversal techniques: DFS and BFS.
2. Learn how DFS and BFS operate on different graph structures.
3. Explore real-world applications: connected components, shortest paths, and cycle detection.

1. Graph Basics

- **Graph:** A collection of vertices (nodes) connected by edges.
- **Types of Graphs:**
 - **Directed Graph (Digraph):** Edges have a direction.
 - **Undirected Graph:** Edges are bidirectional.
- **Graph Representation:**
 - **Adjacency Matrix:** A square matrix where the element at row i and column j indicates the presence of an edge between vertex i and j . Symmetric for undirected graphs.
 - **Adjacency List:** Each vertex has a list of its adjacent vertices. Preferred for sparse graphs due to space efficiency.

2. Depth First Search (DFS)

Concept:

- DFS explores as far as possible along each branch before backtracking.
- Typically implemented using recursion or an explicit stack.

Procedure:

1. Start from a chosen root node.
2. Mark the node as visited.
3. Visit all unvisited neighbors recursively.
4. Backtrack after exploring all neighbors.

Complexity:

Time Complexity: $O(V + E)$ (V = vertices, E = edges)

Space Complexity: $O(V)$ (stack/visited set)

3. Breadth First Search (BFS)

Concept:

- BFS explores all nodes at the current depth level before moving to the next level.
- Typically uses a **queue** to manage node exploration order.

Procedure:

1. Start from a root node.
2. Mark the node as visited and enqueue it.

3. Dequeue a node, process it, and enqueue all unvisited neighbors.
4. Repeat until the queue is empty.

Complexity:

Time Complexity: $O(V + E)$

Space Complexity: $O(V)$ (queue + visited set)

4. DFS vs BFS Observations

Feature	DFS	BFS
Exploration	Goes deep along a branch first	Explores all neighbors at current depth first
Data Structure	Stack (implicit via recursion)	Queue
Use Cases	Cycle detection, topological sorting, maze solving	Shortest path, level order traversal, flood fill
Traversal Order	May vary; depends on recursive path chosen	Level by level; consistent across runs

LAB TASKS:

Cycle Detection in DFS:

1. Modify the DFS implementation to check for cycles in both directed and undirected graphs.
2. Hint: Keep track of the current node's parent in the DFS recursion.

```

import networkx as nx
import matplotlib.pyplot as plt
G = nx.Graph()
edges = [(0,1), (1,2), (2,3), (3,1)]
G.add_edges_from(edges)
def dfs_cycle_undirected(G, node, visited, parent):
    visited.add(node)
    for neighbor in G.neighbors(node):
        if neighbor not in visited:
            if dfs_cycle_undirected(G, neighbor, visited, node):
                return True
        elif neighbor != parent:
            return True
    return False
def has_cycle_undirected(G):
    visited = set()
    for node in G.nodes():
        if node not in visited:
            if dfs_cycle_undirected(G, node, visited, None):
                return True
    return False
print("Cycle exists (Undirected):", has_cycle_undirected(G))
pos = nx.spring_layout(G)
nx.draw(G, pos, with_labels=True,
        node_color='lightblue',
        node_size=700,
        font_weight='bold',
        edge_color='gray')

plt.title("Undirected Graph")

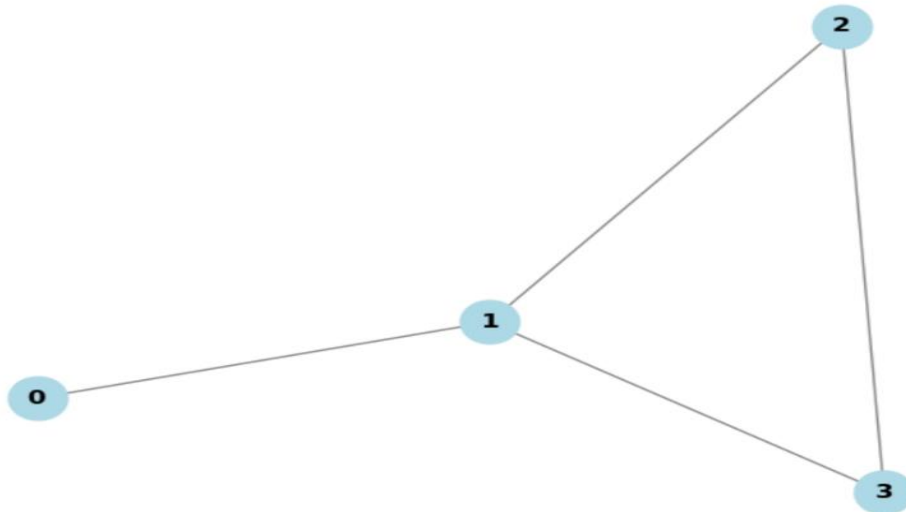
```

3.

```
plt.title("Undirected Graph")
plt.show()

Cycle exists (Undirected): True
```

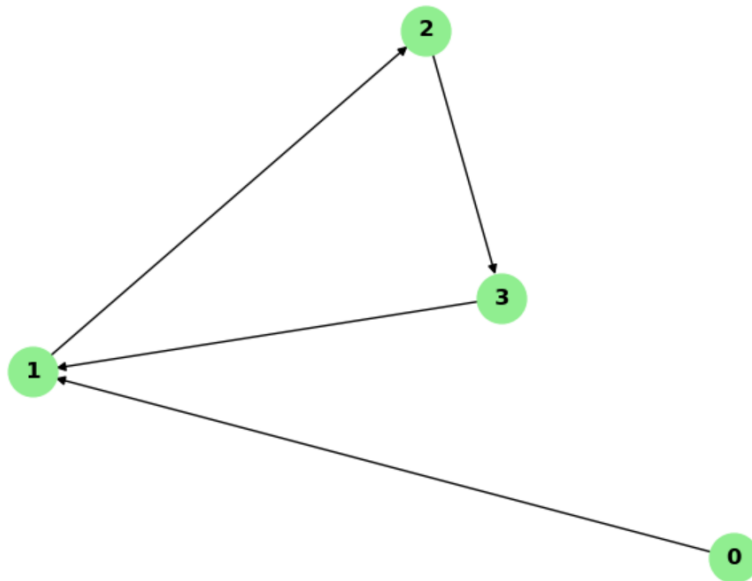
Undirected Graph



```
: import networkx as nx
import matplotlib.pyplot as plt
DG = nx.DiGraph()
edges = [(0,1), (1,2), (2,3), (3,1)]
DG.add_edges_from(edges)
def dfs_cycle_directed(G, node, visited, rec_stack):
    visited.add(node)
    rec_stack.add(node)
    for neighbor in G.neighbors(node):
        if neighbor not in visited:
            if dfs_cycle_directed(G, neighbor, visited, rec_stack):
                return True
        elif neighbor in rec_stack:
            return True
    rec_stack.remove(node)
    return False
def has_cycle_directed(G):
    visited = set()
    rec_stack = set()
    for node in G.nodes():
        if node not in visited:
            if dfs_cycle_directed(G, node, visited, rec_stack):
                return True
    return False
print("Cycle exists (Directed):", has_cycle_directed(DG))
pos = nx.spring_layout(DG)
nx.draw(DG, pos, with_labels=True,
        node_color='lightgreen',
        node_size=700,
        font_weight='bold',
        edge_color='black')
```

Cycle exists (Directed): True

Directed Graph



```
rec_stack.remove(node)
return False
def has_cycle_directed(G):
    visited = set()
    rec_stack = set()
    for node in G.nodes():
        if node not in visited:
            if dfs_cycle_directed(G, node, visited, rec_stack):
                return True
    return False
print("Cycle exists (Directed):", has_cycle_directed(DG))
pos = nx.spring_layout(DG)
nx.draw(DG, pos, with_labels=True,
        node_color='lightgreen',
        node_size=700,
        font_weight='bold',
        edge_color='black')
plt.title("Directed Graph")
plt.show()
```

Shortest Path in Unweighted Graph using BFS:

Modify the BFS to return the shortest path between two nodes.

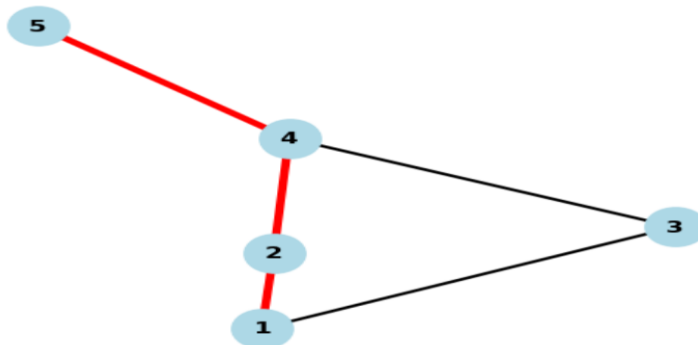
```
import networkx as nx
from collections import deque
import matplotlib.pyplot as plt
G = nx.Graph()
edges = [(1,2), (1,3), (2,4), (3,4), (4,5)]
G.add_edges_from(edges)
def bfs_shortest_path(graph, start, goal):
    queue = deque([start])
    visited = {start: None}
    while queue:
        current = queue.popleft()
        if current == goal:
            path = []
            while current is not None:
                path.append(current)
                current = visited[current]
            path.reverse()
            return path
        for neighbor in graph.neighbors(current):
            if neighbor not in visited:
                visited[neighbor] = current
                queue.append(neighbor)
    return None
start_node = 1
end_node = 5
path = bfs_shortest_path(G, start_node, end_node)
print("Shortest path:", path)
pos = nx.spring_layout(G)
nx.draw_networkx_nodes(G, pos, node_color='lightblue', node_size=800)
```

```
    return path
    for neighbor in graph.neighbors(current):
        if neighbor not in visited:
            visited[neighbor] = current
            queue.append(neighbor)

    return None
start_node = 1
end_node = 5
path = bfs_shortest_path(G, start_node, end_node)
print("Shortest path:", path)
pos = nx.spring_layout(G)
nx.draw_networkx_nodes(G, pos, node_color='lightblue', node_size=800)
nx.draw_networkx_edges(G, pos, width=2)
nx.draw_networkx_labels(G, pos, font_size=12, font_weight='bold')
if path:
    path_edges = list(zip(path, path[1:]))
    nx.draw_networkx_edges(G, pos, edgelist=path_edges, width=4, edge_color='red')
plt.title("Graph with Shortest Path Highlighted")
plt.axis('off')
plt.show()
```

Shortest path: [1, 2, 4, 5]

Graph with Shortest Path Highlighted



Handling Disconnected Graphs:

Modify the DFS and BFS implementations to handle disconnected graphs by starting traversal at each unvisited node.

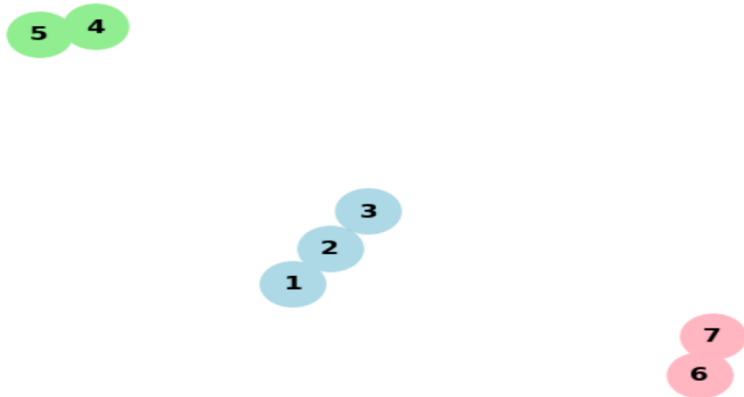
```
]: import networkx as nx
    from collections import deque
    import matplotlib.pyplot as plt
    G = nx.Graph()
    edges = [(1,2), (2,3), (4,5), (6,7)]
    G.add_edges_from(edges)
    def bfs_disconnected(graph):
        visited = set()
        all_components = []
        for node in graph.nodes():
            if node not in visited:
                queue = deque([node])
                component = []
                while queue:
                    current = queue.popleft()
                    if current not in visited:
                        visited.add(current)
                        component.append(current)
                        for neighbor in graph.neighbors(current):
                            if neighbor not in visited:
                                queue.append(neighbor)
                all_components.append(component)
        return all_components
    bfs_components = bfs_disconnected(G)
    print("BFS Components:", bfs_components)
    pos = nx.spring_layout(G)
    colors = ['lightblue', 'lightgreen', 'lightpink', 'yellow']
```

```
        all_components.append(component)
    return all_components
bfs_components = bfs_disconnected(G)
print("BFS Components:", bfs_components)
pos = nx.spring_layout(G)
colors = ['lightblue', 'lightgreen', 'lightpink', 'yellow']
for i, comp in enumerate(bfs_components):
    nx.draw_networkx_nodes(G, pos,
                           nodelist=comp,
                           node_color=colors[i % len(colors)],
                           node_size=800)
nx.draw_networkx_edges(G, pos, width=2)
nx.draw_networkx_labels(G, pos, font_size=12, font_weight='bold')
plt.title("BFS - Disconnected Graph Components")
plt.axis('off')
plt.show()
```

BFS Components: [[1, 2, 3], [4, 5], [6, 7]]

BFS Components: [[1, 2, 3], [4, 5], [6, 7]]

BFS - Disconnected Graph Components

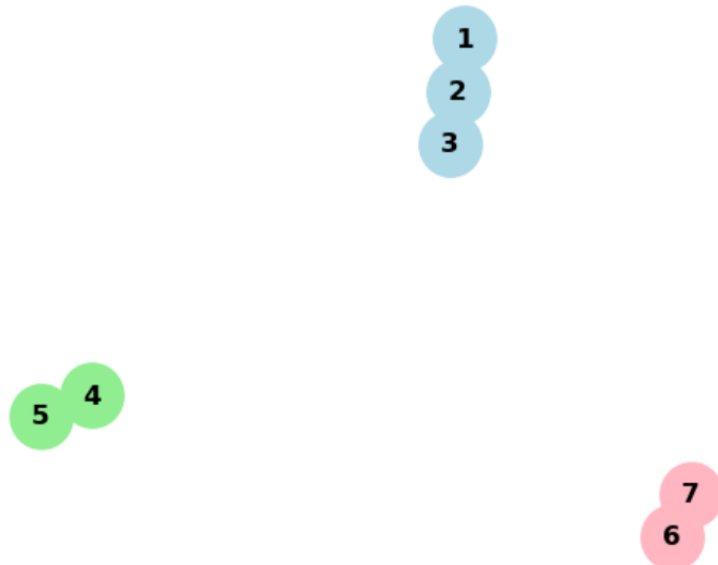


```
]]: import networkx as nx
import matplotlib.pyplot as plt
G = nx.Graph()
edges = [(1,2), (2,3), (4,5), (6,7)]
G.add_edges_from(edges)
def dfs_disconnected(graph):
    visited = set()
    all_components = []
    def dfs(node, component):
        visited.add(node)
        component.append(node)
        for neighbor in graph.neighbors(node):
            if neighbor not in visited:
                dfs(neighbor, component)
    for node in graph.nodes():
        if node not in visited:
            component = []
            dfs(node, component)
            all_components.append(component)
    return all_components
dfs_components = dfs_disconnected(G)
print("DFS Components:", dfs_components)
pos = nx.spring_layout(G)
colors = ['lightblue', 'lightgreen', 'lightpink', 'yellow']
for i, comp in enumerate(dfs_components):
    nx.draw_networkx_nodes(G, pos,
                           nodelist=comp,
                           node_color=colors[i % len(colors)],
                           node_size=800)
```

```
node_size=800)
nx.draw_networkx_edges(G, pos, width=2)
nx.draw_networkx_labels(G, pos, font_size=12, font_weight='bold')
plt.title("DFS - Disconnected Graph Components")
plt.axis('off')
plt.show()
```

DFS Components: [[1, 2, 3], [4, 5], [6, 7]]

DFS - Disconnected Graph Components



LAB #06

Uniform Cost Search (UCS) & Bidirectional Search

Objective

1. To implement and understand Uniform Cost Search (UCS) and Bidirectional Search algorithms.
2. To analyze their working and efficiency in terms of pathfinding and search space reduction.

1. Uniform Cost Search (UCS)

1.1 Introduction

- UCS is an **uninformed search algorithm** used to find the **least-cost path** between a start node and a goal node.
- Unlike heuristic-based searches like A*, UCS does **not use heuristics**.
- UCS explores nodes based on their **cumulative cost from the start**, always expanding the **least-cost node first**.

1.2 Algorithm Steps

1. Initialize a **priority queue** with the start node, prioritized by cumulative path cost.
2. Remove the node with the **lowest cumulative cost** from the queue.
3. If this node is the **goal**, return the path and cost.
4. Otherwise, **expand the node**, adding all adjacent nodes to the queue with their cumulative costs.
5. Repeat until the goal is found or the queue becomes empty.

1.3 Key Concepts

- **Priority Queue:** Maintains nodes based on the **total cost from the start**.
- **Visited Set:** Ensures nodes are not unnecessarily revisited.
- **Optimality:** UCS guarantees the **least-cost path** in a weighted graph with non-negative costs.

1.4 Observations and Results

- UCS **ensures the least-cost path** is found.
- May explore **more nodes** than heuristic searches, but **no node is skipped**.
- Useful in **robotics, route planning, and path finding** problems.

2. Bidirectional Search

2.1 Introduction

- Bidirectional Search simultaneously searches **forward from the start node** and **backward from the goal node**.
- The search stops when the two frontiers **meet in the middle**, reducing the search space.

2.2 Key Concepts

- Instead of searching the entire graph from start, the problem is **split into two searches**: start → middle, and goal → middle.
- Effective for **large graphs**, where search space reduction is critical.

2.3 Algorithm Steps

1. Initialize two **queues**: one for forward search (from start) and one for backward search (from goal).
2. Alternate **BFS expansion** from both queues.
3. When a node is visited by both searches, the algorithm stops, and the **path is reconstructed**.

2.4 Advantages

- **Reduces the number of nodes explored** compared to unidirectional BFS or UCS.
- Can significantly **improve search efficiency** for large graphs.

2.5 Observations and Results

- Bidirectional Search reduces search space and improves time efficiency.
- Path found depends on the **meeting point of the two searches**.
- Useful in **network routing, AI game searches, and shortest path problems**.

LAB TASKS:

Implementation of Uniform cost search

```

•[4]: import heapq
import networkx as nx
import matplotlib.pyplot as plt
G = nx.Graph()
edges = [
    (0, 1, 3),
    (0, 2, 1),
    (1, 2, 3),
    (1, 3, 9),
    (2, 5, 7),
    (5, 4, 8),
    (5, 6, 2),
    (3, 6, 3)
]
G.add_weighted_edges_from(edges)
def uniform_cost_search(graph, start, goal):
    priority_queue = []
    heapq.heappush(priority_queue, (0, start, [start]))
    visited = set()
    nodes_explored = 0
    while priority_queue:
        cost, current_node, path = heapq.heappop(priority_queue)
        nodes_explored += 1
        if current_node == goal:
            return path, cost, nodes_explored
        if current_node not in visited:
            visited.add(current_node)
            for neighbor in graph.neighbors(current_node):
                weight = graph[current_node][neighbor]['weight']
                if neighbor not in visited:

```

```

        for neighbor in graph.neighbors(current_node):
            weight = graph[current_node][neighbor]['weight']
            if neighbor not in visited:
                new_cost = cost + weight
                heapq.heappush(priority_queue,
                               (new_cost, neighbor, path + [neighbor]))

    return None, float('inf'), nodes_explored
start_node = 0
goal_node = 6

path, total_cost, explored_nodes = uniform_cost_search(G, start_node, goal_node)
print("Start Node:", start_node)
print("Goal Node:", goal_node)
print("Optimal Path:", path)
print("Minimum Cost:", total_cost)
print("Nodes Explored:", explored_nodes)
pos = nx.spring_layout(G)
plt.figure(figsize=(8, 6))
nx.draw(G, pos, with_labels=True,
        node_size=2000,
        node_color="lightblue",
        font_size=12)
edge_labels = nx.get_edge_attributes(G, 'weight')
nx.draw_networkx_edge_labels(G, pos, edge_labels=edge_labels)
if path:
    path_edges = list(zip(path, path[1:]))
    nx.draw_networkx_edges(G, pos,
                           edgelist=path_edges,
                           width=4)
plt.title("Uniform Cost Search - Optimal Path Highlighted")

```

```

        edgelist=path_edges,
        width=4)
plt.title("Uniform Cost Search - Optimal Path Highlighted")
plt.show()

```

```

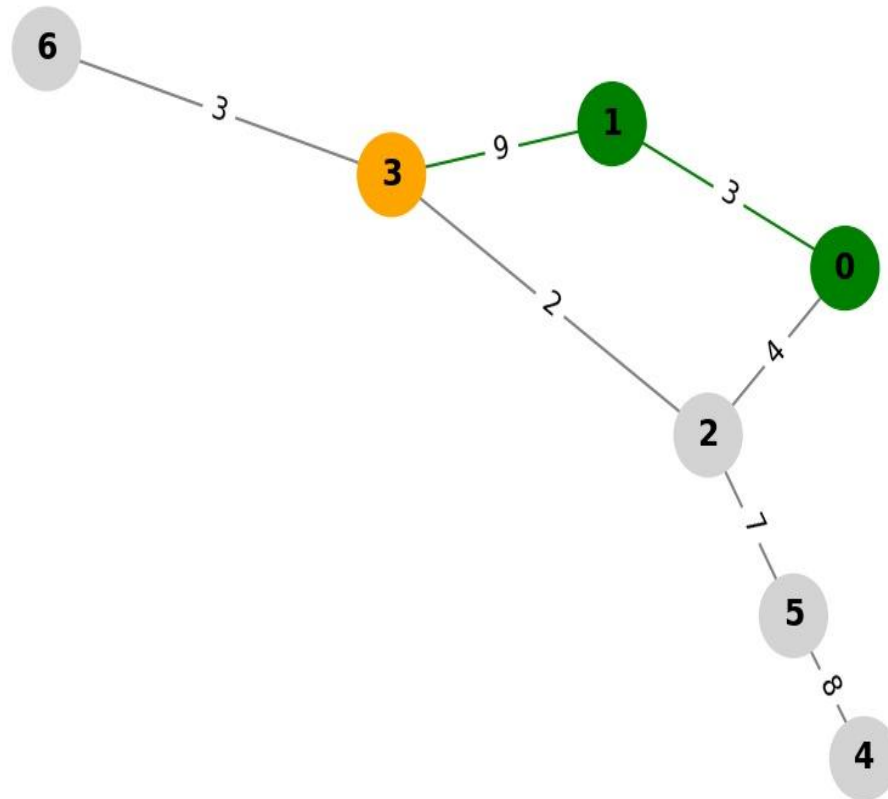
Start Node: 0
Goal Node: 6
Optimal Path: [0, 2, 5, 6]
Minimum Cost: 10
Nodes Explored: 6

```

Uniform Cost Search - Optimal Path Highlighted

Implementation of Bi-directional Search

Bidirectional Search



```

•[3]: import networkx as nx
import matplotlib.pyplot as plt
from collections import deque
G = nx.Graph()
edges = [
    (0, 1, 3),
    (0, 2, 4),
    (1, 2, 3),
    (1, 3, 9),
    (2, 5, 7),
    (5, 4, 8),
    (5, 6, 2),
    (3, 6, 3)
]

G.add_weighted_edges_from(edges)
def bidirectional_search(graph, start, goal):
    if start == goal:
        return [start], 1
    forward_queue = deque([start])
    backward_queue = deque([goal])
    visited_forward = {start: None}
    visited_backward = {goal: None}
    nodes_explored = 0
    while forward_queue and backward_queue:
        current = forward_queue.popleft()
        nodes_explored += 1
        for neighbor in graph.neighbors(current):
            if neighbor not in visited_forward:
                visited_forward[neighbor] = current
                forward_queue.append(neighbor)

            if neighbor not in visited_backward:
                visited_backward[neighbor] = current
                backward_queue.append(neighbor)
            if neighbor in visited_backward:
                return construct_path(neighbor, visited_forward, visited_backward), nodes_explored
        return None, nodes_explored
    def construct_path(meeting_node, visited_forward, visited_backward):
        path_forward = []
        node = meeting_node
        while node is not None:
            path_forward.append(node)
            node = visited_forward[node]
        path_forward.reverse()
        path_backward = []
        node = visited_backward[meeting_node]
        while node is not None:
            path_backward.append(node)
            node = visited_backward[node]

        return path_forward + path_backward
    start_node = 0
    goal_node = 6
    path, explored_nodes = bidirectional_search(G, start_node, goal_node)

```

```

goal_node = 6
path, explored_nodes = bidirectional_search(G, start_node, goal_node)
print("Start Node:", start_node)
print("Goal Node:", goal_node)
print("Path Found:", path)
print("Nodes Explored:", explored_nodes)
pos = nx.spring_layout(G)
plt.figure(figsize=(8, 6))
nx.draw(G, pos,
        with_labels=True,
        node_size=2000,
        node_color="lightgreen",
        font_size=12)

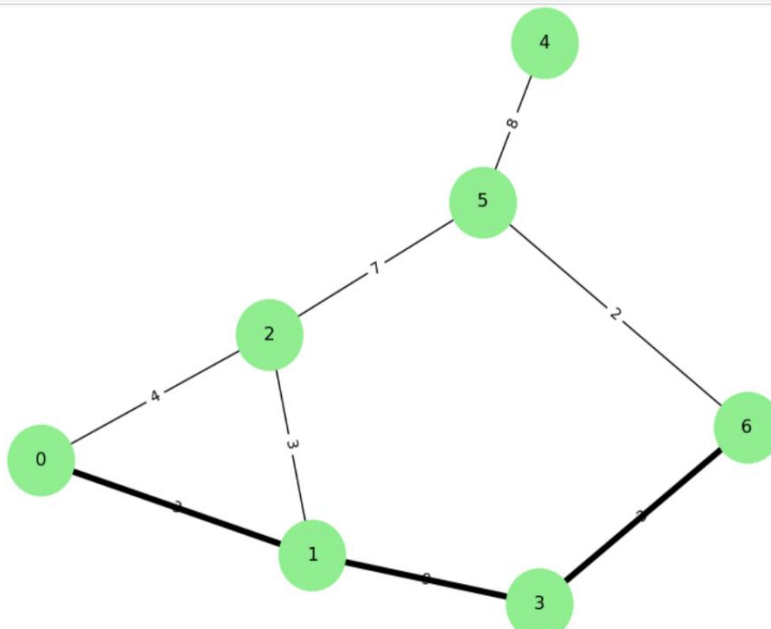
edge_labels = nx.get_edge_attributes(G, 'weight')
nx.draw_networkx_edge_labels(G, pos, edge_labels=edge_labels)
if path:
    path_edges = list(zip(path, path[1:]))
    nx.draw_networkx_edges(G, pos,
                          edgelist=path_edges,
                          width=4)

plt.title("Bidirectional Search - Path Highlighted")
plt.show()

```

Start Node: 0
 Goal Node: 6
 Path Found: [0, 1, 3, 6]
 Nodes Explored: 3

Bidirectional Search - Path Highlighted



Lab 07

Heuristic Search and Best-First Search

Introduction

Artificial Intelligence often requires searching through a large number of possible states to find a solution. Traditional search techniques may explore many unnecessary paths, which increases time and computational cost. Heuristic Search addresses this problem by using additional knowledge about the problem domain to guide the search toward the goal more efficiently. Best-First Search is one of the most commonly used heuristic search algorithms that selects the most promising node based on a heuristic function.

Theory

A heuristic function, denoted by $h(n)$, estimates the cost of reaching the goal state from a given node n . Nodes with smaller heuristic values are considered closer to the goal and are therefore explored first. Best-First Search uses this heuristic information to prioritize node expansion. The algorithm maintains an open list containing nodes to be explored and always selects the node with the lowest heuristic value. This approach often reduces the search space and improves efficiency compared to uninformed search methods.

Methodology

The Best-First Search algorithm was implemented in Python using a priority queue. A graph consisting of nodes and weighted edges was defined, along with heuristic values for each node. Starting from the initial node, the algorithm repeatedly selected the node with the smallest heuristic value, expanded its neighboring nodes, and continued the process until the goal node was reached. Visited nodes were stored to avoid repeated exploration.

Working of the Algorithm

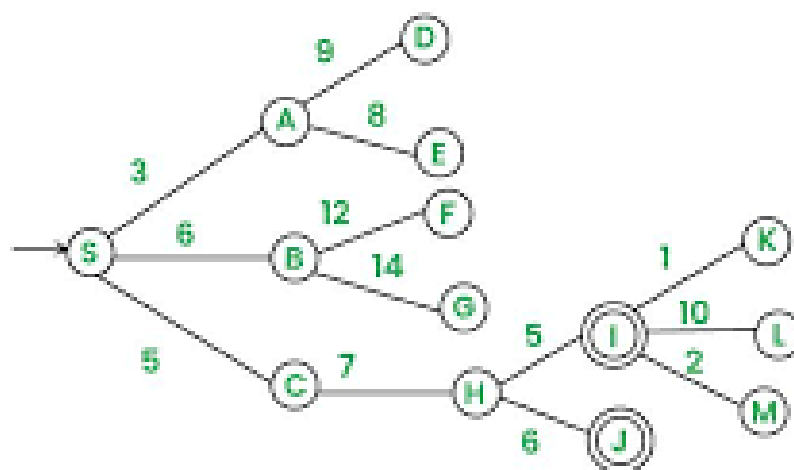
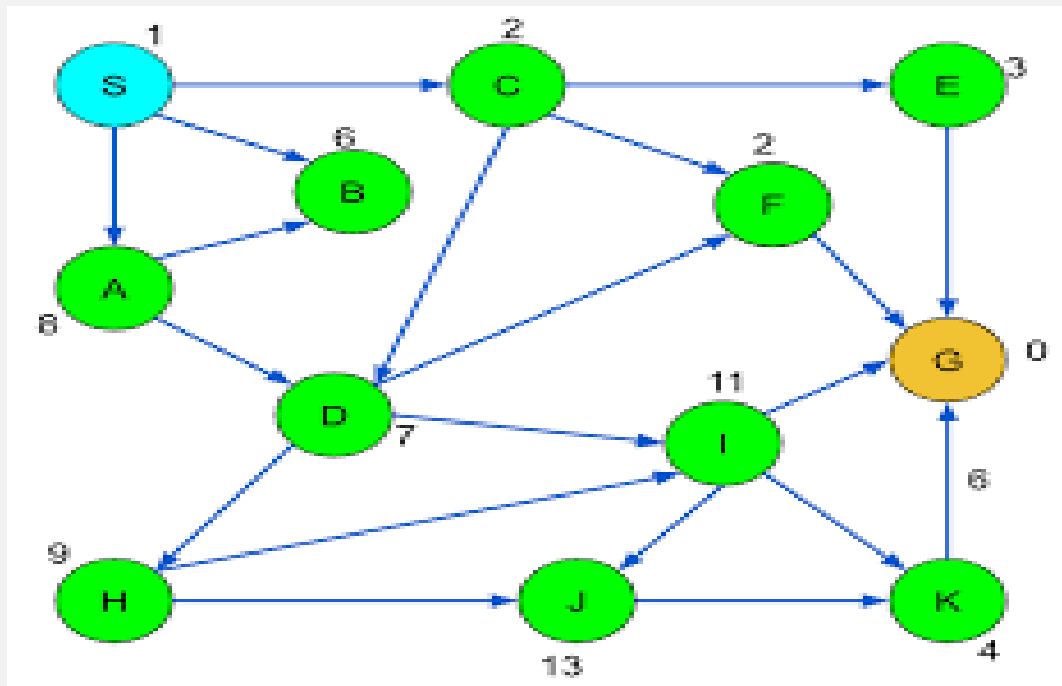
The search begins from the start node S . The algorithm examines the heuristic values of neighboring nodes and always chooses the node that appears closest to the goal. This process continues until the goal node G is found. The priority queue ensures that nodes with lower heuristic values are expanded before others, making the search more directed and efficient.

Results and Discussion

The implementation successfully reached the goal node by following the most promising path according to the heuristic values. The output demonstrated that the algorithm can efficiently guide the search process toward the goal without exploring all possible nodes. However, since Best-First Search relies only on heuristic values, it may not always produce the shortest path. The quality of the solution depends heavily on the accuracy of the heuristic function.

Conclusion

The experiment demonstrated the implementation and working of the Best-First Search algorithm. The use of heuristic information significantly improved search efficiency by directing the search toward the goal. The lab helped in understanding how heuristic functions influence search behavior and highlighted the advantages and limitations of Best-First Search in solving Artificial Intelligence problems.



Lab 08

Unbeatable Tic-Tac-Toe Game

Introduction

Tic-Tac-Toe is a classic two-player strategy game played on a 3×3 grid. In Artificial Intelligence, it is commonly used to demonstrate decision-making and game-playing algorithms. The objective of this lab is to design and implement an unbeatable Tic-Tac-Toe game in which a human player competes against an AI-controlled opponent. The AI uses the Minimax algorithm to evaluate possible moves and make optimal decisions.

Theory

The Minimax algorithm is a decision-making algorithm used in game theory and Artificial Intelligence for two-player games. It works by recursively exploring all possible future game states and selecting the move that maximizes the AI's chances of winning while minimizing the opponent's chances. Each terminal state is assigned a score: a positive score for an AI win, a negative score for a user win, and zero for a draw. By analyzing all possible outcomes, the AI is able to make optimal moves and become unbeatable. Alpha-Beta pruning can further improve efficiency by eliminating branches that cannot influence the final decision.

Methodology

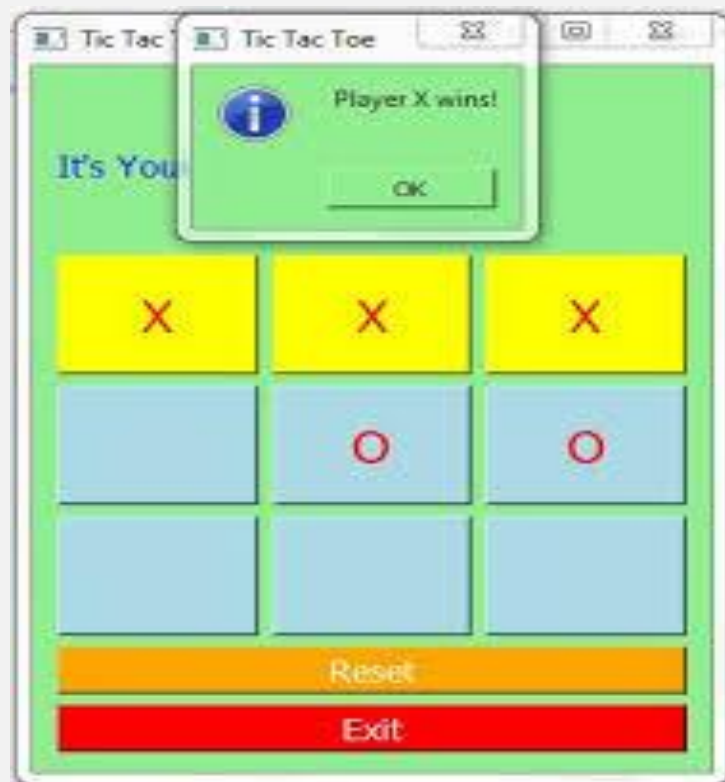
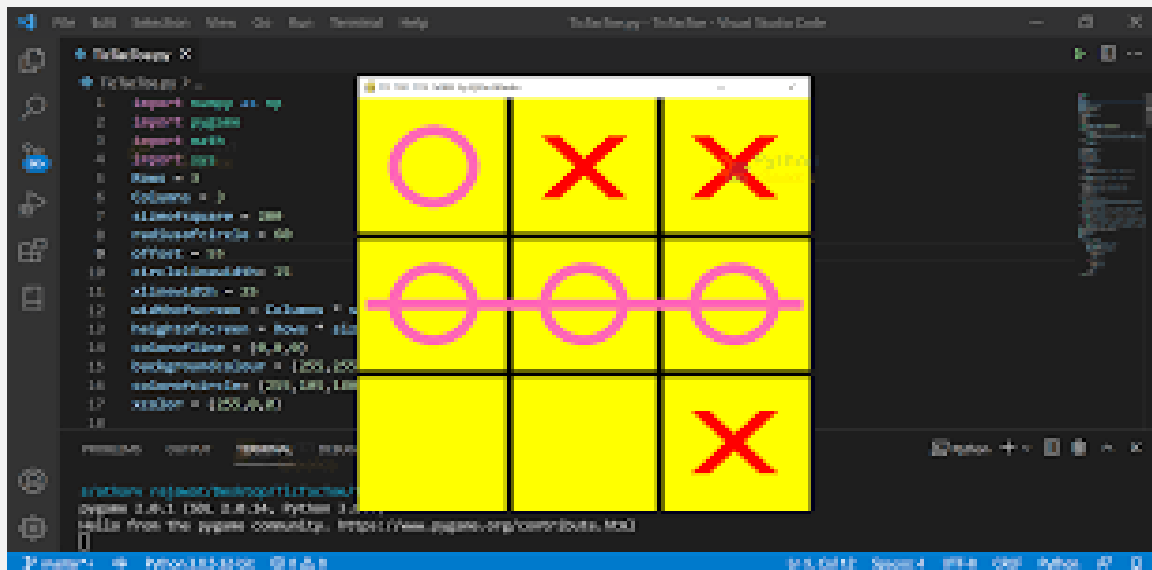
The Tic-Tac-Toe game was implemented using a 3×3 game board. The user and AI take turns placing their symbols on the board. After each move, the program checks whether a player has won or whether the game has ended in a draw. The AI uses the Minimax algorithm to evaluate all legal moves and choose the best possible action. Input validation was included to prevent invalid or repeated moves. The game continues until a winner is determined or the board becomes full.

Results and Discussion

The implemented game successfully allowed a human player to compete against the AI. The AI consistently selected optimal moves, blocked winning opportunities for the user, and took advantage of any mistakes made by the opponent. Testing showed that the AI either won the game or forced a draw, demonstrating the effectiveness of the Minimax algorithm. The game provided a practical understanding of adversarial search and intelligent decision-making techniques used in Artificial Intelligence.

Conclusion

The objective of developing an unbeatable Tic-Tac-Toe game was successfully achieved. The Minimax algorithm enabled the AI to make optimal decisions and avoid defeat. This lab enhanced understanding of game-tree search, state evaluation, and AI-based decision-making. It also demonstrated how intelligent agents can be designed to perform optimally in competitive environments.



Lab 10

Implementation of Naive Bayes Algorithm for Classification

1. Objective

- ❖ To understand the theoretical foundation of the Naive Bayes Algorithm.
- ❖ To implement the Naive Bayes classifier for a classification problem.
- ❖ To evaluate the classifier using performance metrics such as accuracy, precision, recall, and confusion matrix.

2. Theory

Introduction

Naive Bayes is a supervised machine learning algorithm used for classification tasks. It is based on Bayes' Theorem and assumes that all features are independent of each other. Despite this simple assumption, it performs very well on many real-world problems such as spam detection, sentiment analysis, and document classification.

Bayes' Theorem

$$P(A|B) = \frac{P(B|A) \times P(A)}{P(B)}$$

Where:

- **$P(A|B)$** = Probability of event A given event B.
- **$P(B|A)$** = Probability of event B given event A.
- **$P(A)$** = Prior probability of A.
- **$P(B)$** = Prior probability of B.

Types of Naive Bayes

1. Gaussian Naive Bayes

- ❖ Assumes data follows a normal distribution.
- ❖ Suitable for continuous numerical data.

2. Multinomial Naive Bayes

- ❖ Used for discrete count data.
- ❖ Commonly applied in text classification.

3. Bernoulli Naive Bayes

- ❖ Designed for binary features.
- ❖ Useful when features are either present or absent.

3. Algorithm

1. Load the dataset.
2. Separate features and target variable.
3. Split the dataset into training and testing sets.

4. Train the Naive Bayes classifier.
5. Predict class labels for test data.
6. Evaluate the model using accuracy and confusion matrix.
7. Analyze the results.

4. Observations

- ❖ The Gaussian Naive Bayes classifier successfully classified most Iris flower samples.
- ❖ The model achieved approximately 97–98% accuracy.
- ❖ Only a few samples were misclassified.
- ❖ The algorithm performed well because the Iris dataset contains clear class distinctions and numerical features.

5. Result

The Naive Bayes classifier was implemented successfully using the Iris dataset. The model achieved high classification accuracy and correctly predicted most flower species.

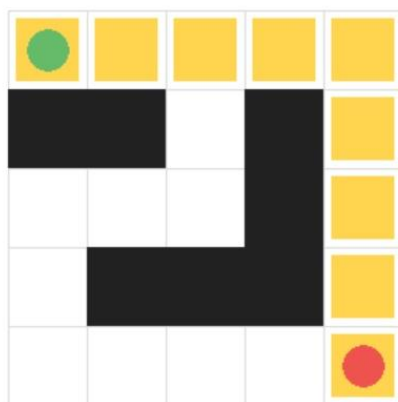
6. Conclusion

The Naive Bayes Algorithm is an efficient and powerful classification technique based on probability theory. It is easy to implement, requires less computational power, and provides good results on many classification problems. Although it assumes independence among features, it still performs remarkably well in practice, especially for text classification, spam filtering, and multiclass prediction tasks.

Therefore, Naive Bayes is a suitable choice when a fast and reliable classification model is required.

Exercise 2 & 3: Maze Search Comparison

Maze Navigation - Algorithm Comparison



Algorithm Stats:

Best-First:	Path: 9 Cost/Nodes: 9/9
A*:	Path: 9 Cost/Nodes: 8/11

Cell Legend:

- Start
- Goal
- Path

Lab 11

Artificial Neural Network (ANN) – Perceptron Model

1. Objective

- ❖ To study the basic concept of Artificial Neural Networks (ANN).
- ❖ To understand the Perceptron Model and its learning algorithm.
- ❖ To implement a perceptron for binary classification.
- ❖ To test the perceptron on logical gate datasets.
- ❖ To analyze the limitations of a single-layer perceptron.

2. Theory

Artificial Neural Network (ANN)

An Artificial Neural Network (ANN) is a computational model inspired by the structure and functioning of the human brain. It consists of interconnected processing units called neurons that work together to solve problems such as classification, prediction, and pattern recognition.

Perceptron Model

The perceptron is the simplest form of an Artificial Neural Network and is used for binary classification tasks. It takes multiple inputs, applies weights, adds a bias term, and passes the result through an activation function to generate an output.

Components of a Perceptron

- 1. Input Layer**
 - Receives input features.
- 2. Weights**
 - Determine the importance of each input.
- 3. Bias**
 - A constant value added to the weighted sum.
- 4. Activation Function**
 - Uses a step function to produce either 0 or 1.

Mathematical Representation

Weighted Sum:

$$[z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b]$$

Activation Function:

$$[y = \begin{cases} 1, & \text{if } z \geq 0 \\ 0, & \text{if } z < 0 \end{cases}]$$

Learning Rule

Weights and bias are updated using:

$$[w = w + \eta (y - \hat{y})x]$$

$$[b = b + \eta (y - \hat{y})]$$

Where:

- ❖ (η) = Learning Rate
- ❖ (y) = Actual Output
- ❖ $(\hat{y})$ = Predicted Output

Experiment 1: Training the Perceptron

Aim

To implement the perceptron learning algorithm using the AND Gate dataset.

Dataset: AND Gate

Feature 1 (x1)	Feature 2 (x2)	Target (y)
0	0	0
0	1	0
1	0	0
1	1	1

Procedure

1. Load the AND gate dataset.
2. Initialize weights and bias.
3. Apply the perceptron learning rule.
4. Update weights whenever prediction is incorrect.
5. Repeat for several epochs.
6. Obtain final weights and bias values.

Source Code

Experiment 2: Testing the Perceptron

Aim

To test the trained perceptron using input samples.

Source Code

Experiment 3: Visualizing the Decision Boundary

Aim

To visualize the decision boundary created by the perceptron.

Procedure

- 1. Plot dataset points.
- 2. Calculate decision boundary using weights and bias.
- 3. Display the graph using Matplotlib.

Source Code

Experiment 4: Limitations of the Perceptron

Aim

To demonstrate the inability of a single-layer perceptron to solve non-linearly separable problems.

Dataset: XOR Gate

Feature 1 (x1)	Feature 2 (x2)	Target (y)
0	0	0
0	1	1
1	0	1
1	1	0

Conclusion

The perceptron is the fundamental building block of Artificial Neural Networks and is useful for binary classification problems involving linearly separable data. It learns by adjusting weights and bias based on prediction errors. Although effective for simple tasks such as the AND gate, it cannot solve non-linearly separable problems like XOR. This limitation led to the development of multi-layer neural networks and deep learning models.

